

ARTIFICIAL INTELLIGENCE

CO5 — Assessment Tool 2 (AT-2): Technical Presentation

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Presentation 1: Reactive Agent-Based Traffic Control System

Designing a Reactive Agent for a smart traffic control system: how the agent responds to real-time traffic inputs, manages signal changes, and ensures minimal congestion using streaming data processing.

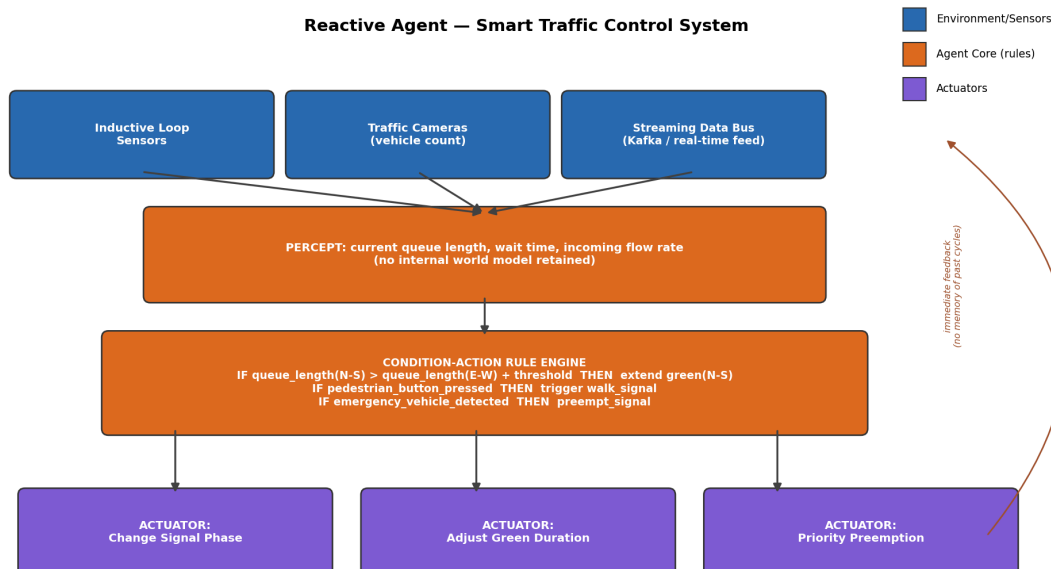
SLIDE 1 Introduction & Problem Statement

- Urban traffic signals controlled by fixed timers cause unnecessary congestion because they cannot react to actual, changing traffic volume at an intersection.
- Goal: design an agent that reacts instantly to live sensor data to minimize average vehicle wait time and queue length.

SLIDE 2 Why a Reactive Agent?

- A reactive agent maps percepts directly to actions using condition-action rules and keeps no internal model of history — perfect for traffic control, where the correct action depends only on the current queue state, not on what happened five minutes ago.
- This keeps the decision latency extremely low, which matters because signal decisions must be made within the current traffic light cycle (seconds), not after a lengthy planning process.

SLIDE 3 System Architecture



SLIDE 4 Perception: Streaming Data Processing

- Inductive loop sensors and traffic cameras continuously publish vehicle counts and queue lengths onto a streaming bus (e.g., Kafka); PySpark Structured Streaming consumes this as a sequence of micro-batch RDDs.
- Each micro-batch is processed independently — consistent with the reactive agent's stateless design — computing `queue_length`, `wait_time`, and `incoming_flow_rate` per lane/direction for the current instant.

SLIDE 5 Condition-Action Rule Engine

- IF `queue_length(N-S) > queue_length(E-W) + threshold` THEN `extend_green(N-S)`
- IF `pedestrian_button_pressed` THEN `trigger_walk_signal`
- IF `emergency_vehicle_detected` THEN `preempt_signal`
- Rules are evaluated in priority order every cycle so higher-priority safety rules (e.g., emergency preemption) always override routine congestion-balancing rules.

SLIDE 6 Signal Management & Congestion Minimization

- Green-time is dynamically extended for the direction with the larger queue and shortened for the lighter direction, rebalancing capacity toward demand every cycle rather than following a fixed split.
- Because decisions are made every cycle from fresh percepts, the system self-corrects quickly if traffic patterns shift (e.g., after an accident or event), which a static timer plan cannot do.

SLIDE 7 Advantages & Limitations

- **Advantages:** Extremely fast response, simple to implement and verify, scales to many independent intersections since each agent only needs local percepts.
- **Limitations:** No memory of past states means the agent cannot anticipate recurring patterns (e.g., daily rush-hour build-up) or coordinate proactively across multiple intersections — this would need a higher-level planning or learning layer on top.

SLIDE 8 Conclusion

- A reactive agent architecture, combined with real-time streaming data processing, is well-suited to traffic signal control because it delivers the low-latency, locally-optimal decisions the problem demands, at the cost of not learning or planning across longer time horizons.

Presentation 2: Goal-Based Agent for Healthcare Diagnosis

A Goal-Based Agent used in an AI healthcare system: how the agent sets diagnostic goals, evaluates patient data, and selects optimal treatment pathways using decision-making logic and data pipelines.

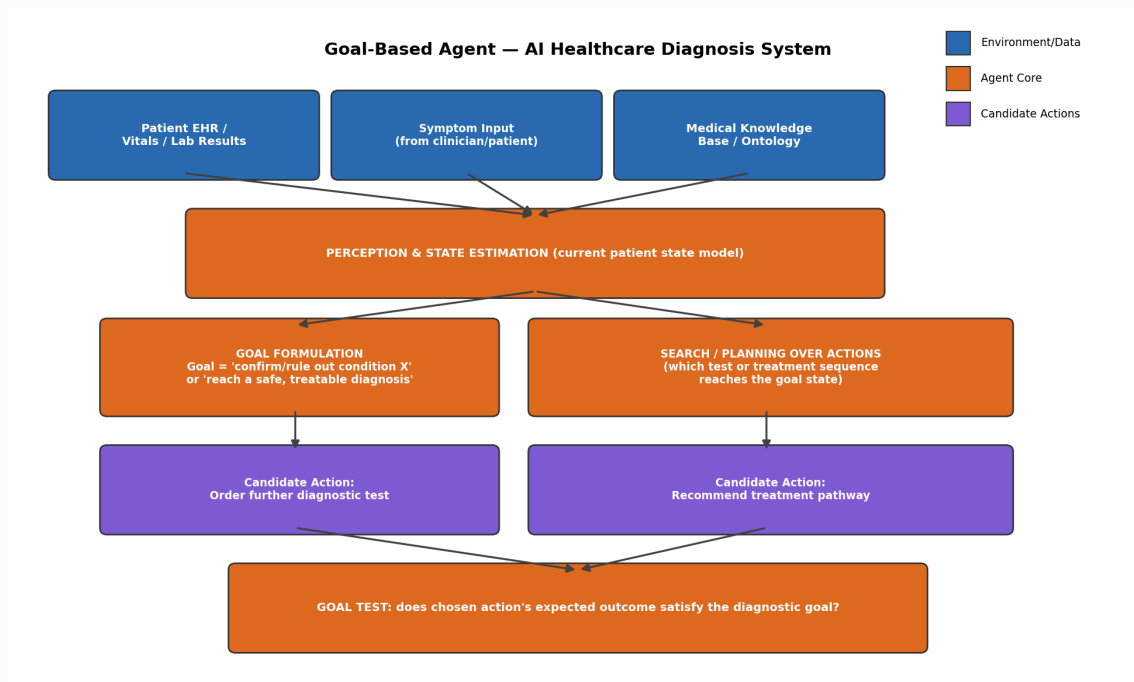
SLIDE 1 Introduction & Problem Statement

- Diagnosis is not a single lookup — it is a process of narrowing down possibilities toward a specific, actionable conclusion while managing risk and cost (e.g., unnecessary tests).
- Goal: design an agent that treats diagnosis as a goal to be achieved through a sequence of reasoned actions, not a fixed reflex.

SLIDE 2 Why a Goal-Based Agent?

- Unlike a reactive agent, a goal-based agent maintains an explicit goal (e.g., 'confirm or rule out condition X') and considers the future consequences of actions before choosing one — essential in healthcare, where actions (tests, treatments) have costs and risks that must be weighed against how much they help reach the diagnostic goal.

SLIDE 3 System Architecture



SLIDE 4 Data Pipeline & Patient State Estimation

- Patient EHR records, live vitals, lab results, and clinician-entered symptoms are ingested into a data pipeline that continuously updates a structured patient-state model (the agent's internal representation of 'what is currently known').
- This state model is what distinguishes the goal-based agent from a reactive one — it persists and accumulates evidence across the whole diagnostic encounter, not just the current percept.

SLIDE 5 **Goal Formulation & Search/Planning**

- Given ambiguous initial symptoms, the agent formulates a specific goal such as 'rule out condition X' or 'reach a diagnosis with sufficient confidence to prescribe safely.'
- The agent then searches over possible sequences of actions (which test to order next, in what order) using the medical knowledge base to predict which action sequence most efficiently reaches the goal state — analogous to search/planning in classical AI, but over a decision tree of tests and outcomes.

SLIDE 6 **Goal Test & Treatment Pathway Selection**

- After each new piece of evidence, a goal test checks whether the current patient-state model satisfies the diagnostic goal with sufficient confidence.
- Once satisfied, the agent selects the treatment pathway whose predicted outcome best matches the confirmed diagnosis and patient-specific constraints (allergies, comorbidities, cost) — treatment selection is itself framed as reaching a second-level goal: 'the patient's condition improves safely.'

SLIDE 7 **Advantages & Limitations**

- **Advantages:** Explicitly reasons about future consequences, can justify its recommended tests/treatments in terms of the goal, adapts its plan as new evidence arrives.
- **Limitations:** Requires an accurate, well-structured knowledge base and can be computationally expensive to search over many possible test/treatment sequences for complex cases; does not by itself weigh competing goals numerically (e.g., cost vs. speed vs. accuracy) — that requires a utility-based extension.

SLIDE 8 **Conclusion**

- Framing diagnosis and treatment selection as goal-directed search over a continuously-updated patient state allows the agent to behave like a careful clinician working systematically toward a conclusion, rather than reacting to isolated symptoms.

Presentation 3: Utility-Based Agent for E-Commerce Recommendations

A Utility-Based Agent that generates personalized product recommendations: how the agent assigns utility values to different options and optimizes user satisfaction using large-scale data processing.

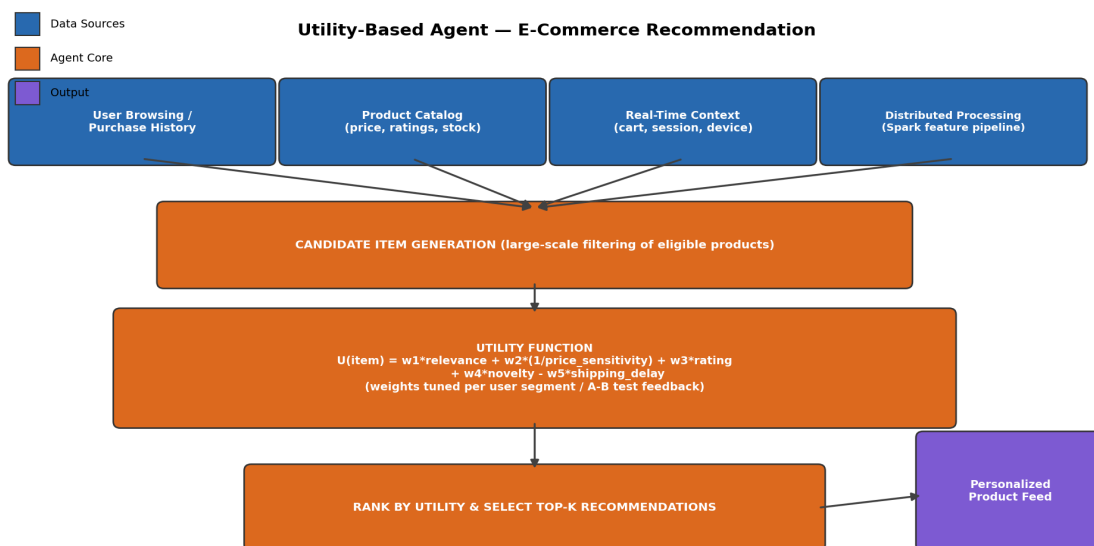
SLIDE 1 Introduction & Problem Statement

- A recommendation is rarely a single 'correct' answer — many products could reasonably be shown, and the platform must choose which ones actually maximise the customer's satisfaction (and, indirectly, business value).
- Goal: design an agent that ranks options by a measurable degree of desirability rather than a binary goal-satisfied/not-satisfied test.

SLIDE 2 Why a Utility-Based Agent?

- A utility-based agent assigns a real-valued utility to each possible outcome and chooses the action that maximises expected utility — appropriate here because 'a good recommendation' is inherently a matter of degree (more relevant, cheaper, better rated) rather than a single goal that is either met or not.

SLIDE 3 System Architecture



SLIDE 4 Large-Scale Data Processing

- User browsing/purchase history, the full product catalog, and real-time session context (cart contents, device) are combined at scale using a distributed feature pipeline (e.g., PySpark), since a large platform must score millions of user-item pairs.
- A first filtering stage narrows the full catalog down to a manageable candidate set (e.g., items in the user's browsed categories, in stock) before the more expensive utility computation runs — this keeps the pipeline scalable.

SLIDE 5 The Utility Function

- $U(\text{item}) = w_1 \cdot \text{relevance} + w_2 \cdot (1/\text{price_sensitivity}) + w_3 \cdot \text{rating} + w_4 \cdot \text{novelty} - w_5 \cdot \text{shipping_delay}$
- Each term captures one dimension of what makes a recommendation good for this specific user; the weights ($w_1 \dots w_5$) are tuned per user segment and continuously refined using A/B test outcomes, so 'satisfaction' is treated as a learned, personalized quantity rather than a fixed formula.

SLIDE 6 Ranking & Recommendation Output

- Every candidate item's utility is computed, items are ranked in descending order of utility, and the top-K items are shown as the personalized product feed.
- Because utility is a single combined score, trade-offs are handled automatically — e.g., a slightly less relevant item with a much better price or rating can still outrank a 'more relevant' but expensive item if its overall utility is higher.

SLIDE 7 Advantages & Limitations

- **Advantages:** Naturally handles trade-offs between competing factors (price, relevance, rating, delay), gives a single interpretable ranking score, and adapts smoothly as weights are retrained.
- **Limitations:** Requires careful weight-tuning and enough behavioural data per user segment to be reliable; a poorly-calibrated utility function can systematically over- or under-value one factor (e.g., over-prioritising price and hurting perceived relevance).

SLIDE 8 Conclusion

- Utility-based reasoning turns 'maximise customer satisfaction' from a vague objective into a concrete, tunable scoring function that can rank any number of candidate products consistently at scale.

Presentation 4: Learning Agent for Personalized Education System

The design of a Learning Agent in an AI-driven educational platform: how the agent improves performance over time using feedback, adapts content difficulty, and integrates with distributed data processing systems like PySpark.

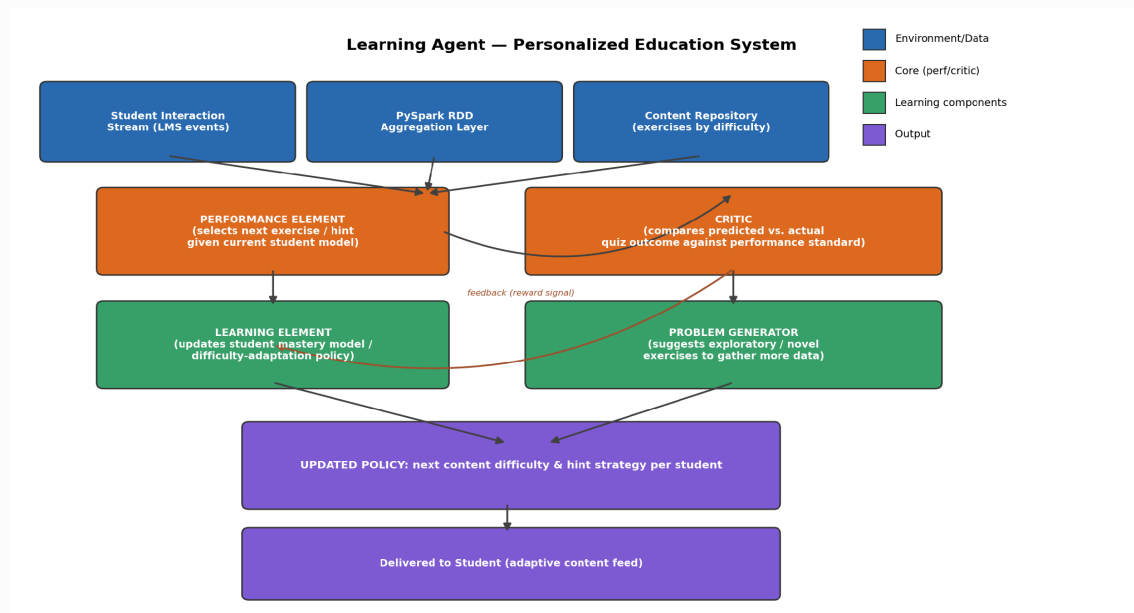
SLIDE 1 Introduction & Problem Statement

- A fixed, one-size-fits-all curriculum either bores advanced students or overwhelms struggling ones; the ideal system continuously adjusts difficulty to each learner's evolving ability.
- Goal: design an agent whose recommendation policy actually improves over time from experience, rather than remaining static after deployment.

SLIDE 2 Why a Learning Agent?

- A learning agent is the only architecture among the four classical types that explicitly improves its own performance element using feedback from a critic — necessary here because the 'right' difficulty level for a given student is not known in advance and must be discovered and refined from observed outcomes.

SLIDE 3 System Architecture (4-component learning agent)



SLIDE 4 Performance Element & Critic

- The Performance Element selects the next exercise, hint, or explanation to present, based on the current student model (estimated mastery per topic).

- The Critic compares the predicted outcome (e.g., expected quiz score) against the actual outcome the student achieves, producing a feedback/reward signal that indicates whether the last decision was well-calibrated.

SLIDE 5 **Learning Element & Problem Generator**

- The Learning Element uses the Critic's feedback to update the student mastery model and the difficulty-adaptation policy — this is where actual improvement over time happens, e.g., recalibrating how quickly to raise difficulty after a correct answer.
- The Problem Generator occasionally suggests exploratory or slightly off-policy exercises (not just the 'safest' next item) so the system keeps gathering informative data about the student's true ability, preventing the model from getting stuck on stale estimates.

SLIDE 6 **Integration with PySpark for Scale**

- Raw clickstream, quiz, and video-engagement events from thousands of concurrent learners are aggregated using PySpark RDD operations (groupByKey by student_id, aggregateByKey for running averages), which is what allows the same learning-agent logic to run per-student at platform scale rather than one student at a time.
- Because RDDs can be partitioned by student_id, each student's model update runs independently and in parallel across the cluster, keeping feedback close to real-time even as enrolment grows.

SLIDE 7 **Advantages & Limitations**

- **Advantages:** Genuinely improves over time without manual re-tuning, personalizes at scale, and its problem generator prevents premature convergence to a suboptimal policy.
- **Limitations:** Requires enough interaction data per student before estimates become reliable (cold-start for new learners), and exploratory exercises from the Problem Generator can occasionally present a student with a slightly mismatched difficulty level in service of long-run learning.

SLIDE 8 **Conclusion**

- The four-component learning-agent architecture, combined with PySpark-based distributed aggregation, lets a personalized education platform continuously refine what it delivers to each learner while scaling to a large, concurrent student population.

Presentation 5: Multi-Agent System for Disaster Management

A Multi-Agent System where multiple agents (sensor agents, decision agents, communication agents) collaborate for disaster detection and response, covering coordination strategies, data sharing, and real-time decision-making.

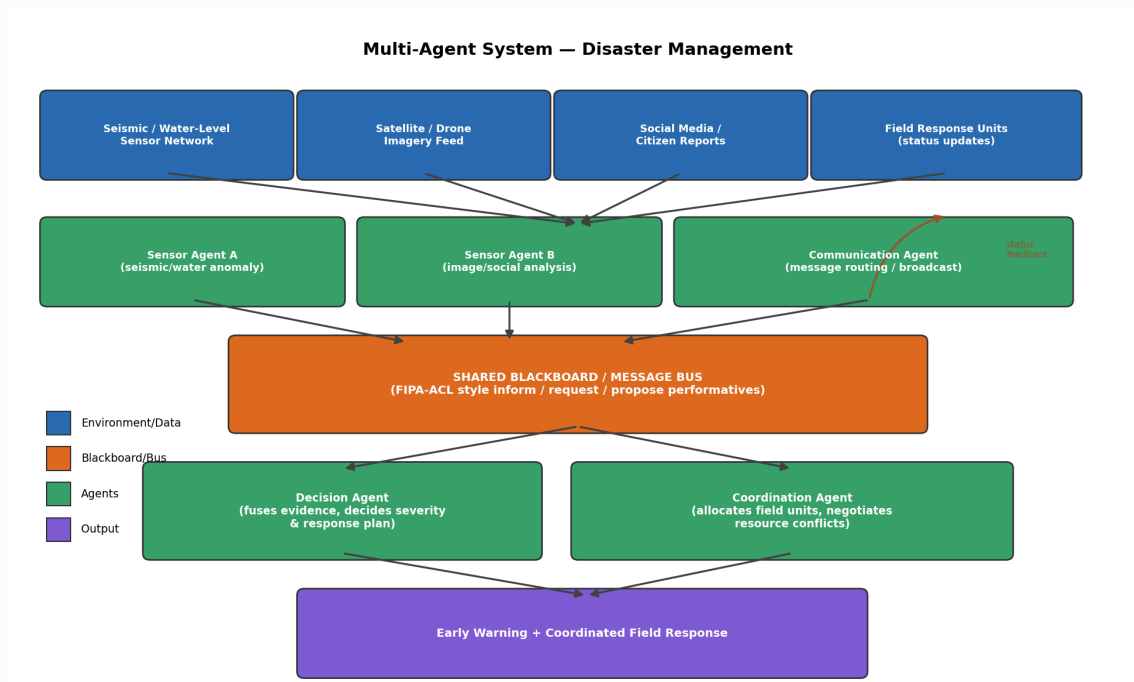
SLIDE 1 Introduction & Problem Statement

- Disaster detection and response naturally spans many specialised sub-tasks — sensing, image analysis, communication with field units, and decision-making — that are best handled by separate, cooperating components rather than one monolithic system.
- Goal: design a multi-agent system (MAS) where specialised agents share evidence and coordinate a response faster and more reliably than any single agent could alone.

SLIDE 2 Why a Multi-Agent System?

- Each data source (sensors, imagery, social reports, field units) requires different expertise to interpret, and the geographic scale of a disaster means processing must be distributed; a MAS lets each agent specialise and operate concurrently while still contributing to one shared decision.

SLIDE 3 System Architecture



SLIDE 4 Agent Roles

- **Sensor Agents:** Sensor Agent A processes seismic/water-level sensor streams for statistical anomalies; Sensor Agent B processes satellite/drone imagery and social-media reports for corroborating visual/textual evidence.
- **Decision Agent:** Fuses evidence from both sensor agents to assess disaster likelihood and severity, and formulates a response plan.
- **Communication Agent:** Routes messages between agents and broadcasts confirmed warnings and instructions to field response units and the public.
- **Coordination Agent:** Allocates available field units to affected zones and resolves resource conflicts when multiple zones compete for the same limited units.

SLIDE 5 **Coordination Strategy & Data Sharing**

- Agents communicate through a shared blackboard / message bus using structured performatives (inform, request, propose — in the style of the FIPA-ACL agent communication standard) rather than direct point-to-point calls, so any agent can publish evidence that any other agent may need, without tight coupling.
- This shared-blackboard pattern is what allows the Decision Agent to fuse independently-generated evidence from multiple sensor agents into one coherent severity assessment, rather than each agent unilaterally deciding to raise an alarm.

SLIDE 6 **Real-Time Decision-Making**

- As new sensor/imagery/social evidence streams in, the Decision Agent continuously re-evaluates severity, and the Coordination Agent adjusts field-unit allocation on the fly (e.g., redirecting units if a second, more severe zone is confirmed) rather than committing to a single static response plan.
- Field units report status back through the Communication Agent, closing the loop so the system's picture of 'what has already been done' stays current and coordination avoids duplicated or conflicting responses.

SLIDE 7 **Advantages & Limitations**

- **Advantages:** Specialisation improves the quality of each sub-task, agents can be added or scaled independently, and shared-blackboard communication is robust to individual agent slowdowns or failures.
- **Limitations:** Requires careful protocol/message design to avoid conflicting instructions between agents, and overall system latency depends on the slowest agent in a decision chain — coordination overhead itself must be kept fast enough for a genuine emergency.

SLIDE 8 **Conclusion**

- A multi-agent architecture with a shared communication layer allows a disaster-management system to combine specialised, concurrently-operating agents into one fast, corroborated early-warning and response capability that a single centralized agent would struggle to match.

